Orca — Prior Art Record

Author: James Yarber (GitHub: junkyard22) **Organization:** YakStacks **Primary Repo:** github.com/junkyard22/Orca — private **Public Release:** Orca v1.0.0 (current version v1.2.2) **Published Spec:** Agent Handoff Protocol (AHP) **Published Runtime:** @marsulta/mailman v0.1.0 **Document Purpose:** Establishing timestamped prior art for key architectural concepts independently conceived and implemented in the Orca project.

Summary

Orca is a local-first multi-agent AI orchestration desktop application for Windows, built in TypeScript/Electron. It was conceived and developed solo by James Yarber, beginning in early 2025, predating the public announcement or release of several well-funded projects that have since shipped individual pieces of Orca's integrated architecture.

The core thesis: **the model is the spigot, the contract is the nozzle.** Output quality is determined at the boundary, not at the source. Any LLM — frontier, local, cheap, expensive — can fulfill a role as long as it passes the contract. Pappy enforces that contract. Moonshiner trains better spigots from verified flows over time.

This document records the architectural concepts, the repositories that contain them, the dates those repositories were created or last updated, and the pattern of well-funded teams independently arriving at individual pieces of the same architecture — all verifiable via public commit history, published packages, arXiv papers, and press coverage.

Repository Timeline (Verified from GitHub)

YakStacks Organization (`github.com/orgs/YakStacks`)

Repository	Visibility	Language	Last Updated	Description
`Moonshiner`	Private	Python	**Nov 29, 2025**	Distillation pipeline
`moonshiner-recipes`	**Public**	**MoonScript**	**Nov 29, 2025**	DSL recipes for distillation
`maestro`	Private	Python	Mar 2026	Original Python orchestration engine ("AI IDE Team")

Repository	Visibility	Language	Last Updated	Description
`Workbench`	**Public**	JavaScript	Feb 2026	Local-First AI Task Runner
`Pipewrench`	**Public**	JavaScript	Mar 2, 2026	MCP Connection Diagnostic & Proxy Tool
`CodePad`	Private	JavaScript	Feb 2, 2026	Mobile companion app
`YakStacks`	Private	TypeScript	Feb 10, 2026	Community platform

junkyard22 Personal (`github.com/junkyard22`)

Repository	Visibility	Language	Notes
`Orca`	Private	TypeScript	186+ commits, tags: ai, mcp, orchestration, multi-agent, orchestrator, ai-agents
`AHP`	**Public**	TypeScript	Agent Handoff Protocol specification
`Mailman`	Private	TypeScript	Transport layer implementation
`AutoScribe`	Private	TypeScript	Narration/transcription component
`clawde`	Private	TypeScript	Provider adapter layer
`Neural-Equalizer-System-`	**Public**	Markdown	Conceived Dec 11, 2025 — separate neurotechnology framework

Published Packages

Package	Version	Registry	Published
`@marsulta/mailman`	0.1.0	npm	Apr 7, 2026 — AHP runtime, three-agent proof of concept verified

---

Core Architectural Thesis

Contracts Over Prompting

Most agent systems attempt to shape behavior through prompting — primary prevention. This is probabilistic and breaks down across model diversity. Every new model requires its own tuning, quirk handling, and adapter logic.

Orca's approach is output validation — secondary prevention. Contracts at the boundary. Pass or fail. Deterministic. The contract does not care which model fulfilled it.

The water hose mental model:

- The **spigot** is the model (Opus, Haiku, Qwen, local jar, whatever)
- The **nozzle** is the output contract (schema, acceptance criteria)
- **Pappy** is the pressure regulator — nothing reaches the garden until it meets spec
- **Moonshiner** studies successful flows and trains jars that hit spec reliably with less water

Most platforms sell a better spigot. Orca sells the nozzle. That is a fundamentally different value proposition and much harder to copy, because it is not about which model you have access to — it is about what must be true for output to be accepted.

Named Components (Architectural Inventory)

Component	Role	Status
Brain	Task decomposition, routing, planning	Shipped in v1.0.0
Benson	Intake/output — the only user-facing voice	Shipped in v1.0.0
Miranda	Budget enforcement, permissions, compliance gating, lifecycle transitions	Shipped in v1.0.0
Pappy	Quality gate / QC verifier (named after a George Jones song, "White Lightning")	Shipped in v1.0.0
Dewey	Persistent user context, preference learning (named after Dewey Decimal)	Shipped in v1.0.0, preference learning active
Moonshiner	Distillation pipeline — exports Pappy-verified JSONL; retrain cycle	Shipped, auto-loop pending

Component	Role	Status
	currently manual	
Narrator	Internal technical summary writer	Shipped in v1.0.0
Maestro	Lifecycle orchestration engine	Shipped in v1.0.0
ShineRunner	Benchmarking / LLM evaluation	Shipped
AHP (Agent Handoff Protocol)	Typed packet-based agent-to-agent communication standard	Published as open spec + npm runtime (`@marsulta/mailman`)
Jars (v2/v3 vision)	Small specialist models trained from Pappy-verified runs, swappable at orchestration layer	Architecture defined, RunPod path identified for first distillation (Python specialist)

Key Concepts & Prior Art Dates

1. Quality-Gated Self-Improvement Loop (Pappy → Moonshiner)

The concept: A distillation pipeline that only accepts training data from quality-verified runs. An LLM quality verifier (Pappy) gates what enters the training pipeline (Moonshiner). This is distinct from self-reported quality, judge-based post-hoc consensus, or raw trajectory collection.

Prior art:

- `YakStacks/Moonshiner` — Python — active as of **Nov 29, 2025**
- `YakStacks/moonshiner-recipes` — **public** MoonScript DSL repo — **Nov 29, 2025**
- Stages documented: Mash → Still → Radiator → Barrel → Proof → Bottle
- Custom DSL (`mash_bill`, `still`, `barrel_age`, `bottle`) written specifically for recipe definition

What shipped later:

- **MiniMax M2.7** — self-evolving loop — March 2026 (no quality gate documented)
- **Hermes Agent (Nous Research)** — self-training via Atropos RL framework — February 2026 (trajectory collection without verification gate)
- **Memento-Skills** (arXiv:2603.18743) — "Let Agents Design Agents" — Agents generating skills for other agents, no quality gate between generation and ingestion
- **Cursor Composer 2** — self-summarization / distillation features

- **Qualixar OS** (arXiv:2604.06392, April 7, 2026) — self-improvement loop benchmark showed scores *declining* across iterations with p=0.578 — statistically insignificant. This is exactly the failure mode a Pappy-style gate prevents.
- **Qodo** (webinar, 2026) — automated quality gates + "living governance system that learns"

Orca's differentiator: Pappy verifies runs *before* they enter Moonshiner. No other public system has documented this contractual relationship between a quality verifier and a distillation pipeline prior to November 29, 2025. The Qualixar benchmark result is direct evidence of what happens without this gate: the training signal degrades.

---

2. Specialist Model Routing ("Jars") with Transparent Swapping

The concept: Rather than routing between LLM providers (GPT vs Claude), route between verified specialist roles exposed to the orchestration layer as interchangeable slots. Jar tiers: local (9B, on-device), community (shared recipes), cloud (subscription), enterprise (domain fine-tunes). Users can swap jar versions without changing how the system works — "the software update model applied to intelligence."

Prior art:

- `YakStacks/maestro` — Python — "AI IDE Team"
- `junkyard22/Orca` — TypeScript rewrite, 186+ commits
- Jar tier architecture documented
- `YakStacks/moonshiner-recipes` — the public recipe repo is the infrastructure for community jar sharing
- First planned jar: Python specialist fine-tuned from DeepSeek-Coder or Gemma 4

What shipped later:

- **MiniMax M2.7** — "self-evolving capabilities" — March 2026
- **Hermes Agent** — "autonomous skill creation" — February 2026
- **Claude Code** (screenshot captured April 24, 2026) — visible multi-model routing: "Running agent Haiku 4.5 Bash" while Opus handles reasoning. Specialized agents, different models for different roles, orchestrated together. Anthropic's own product shipping the jar pattern as a user-facing feature.
- **Google Antigravity** — parallel agents in a coding IDE
- **NeoCognition** (TechCrunch, April 21, 2026) — raised **$40M seed** to build "agents that self-learn to become experts in any domain" via "world models for any micro-environment" — co-led by Cambium Capital and Walden Catalyst Ventures, with Intel CEO Lip-Bu Tan and

Databricks co-founder Ion Stoica participating. 15 employees, mostly PhDs. Selling to enterprises. The specialization thesis is identical to jars + Dewey.

Orca's differentiator: Jars are trained from Pappy-verified runs via Moonshiner. Specialization and quality gating are a single integrated loop, not separate research problems. NeoCognition raised $40M to build the specialization piece. Orca has the foundation committed to version control and the integration with the quality layer already mapped.

---

3. Compliance / Permissions Gate Before Tool Execution (Miranda)

The concept: A dedicated compliance agent that enforces behavioral rules, budget constraints, and permission scoping *before* any tool is executed. Separate from quality verification (Pappy). Applies to MCP tools as well as native tools.

Prior art:

- `junkyard22/Orca` — Miranda component, TypeScript — active in commit history predating Feb 2026
- `feat(mcp): wire Desktop Commander and GitHub MCP server...` — Miranda's tool filtering explicitly applied to MCP tools
- `before_qc` gate documented in pipeline trace logs

What shipped later:

- **GitAgent** — March 2026 — "Docker for AI agents" — tool approval gating before execution
- **OpenClaw** security failures (CVE-2026-25253) — March 2026 — supply chain attacks through an open skill marketplace. Exactly the failure mode Miranda's gate is designed to prevent.
- **Anthropic Claude Managed Agents** (Wired, April 2026) — cloud-hosted sandboxed execution with scoped permissions and credential management. The pattern Miranda implements, productized as platform infrastructure at **$0.08 per session hour** cloud tax. Practitioner pushback already surfacing about lock-in, no open standard, no portability.

Orca's differentiator: Miranda runs locally. No cloud tax. No vendor lock-in. No SDK dependency. Enforcement is a contract, not a service subscription.

---

4. User Context Librarian with Persistent Behavioral Learning (Dewey)

The concept: A dedicated agent responsible for user context, behavioral observations, and pre-flight briefing of the orchestration pipeline. Named after the Dewey Decimal system. Learns user

preferences over time. Future milestone uses Moonshiner to compress raw behavioral observations into compact warm context facts.

Prior art:

- `junkyard22/Orca` — Dewey component — TypeScript — active in commit history
- `userContext.json` at `~/.orca/userContext.json` populated and verified in tracer tests (session logs, March 2026)
- Dewey context compression via Moonshiner identified as future milestone

What shipped later:

- **Hermes Agent v0.6.0** — persistent memory via SQLite + FTS5 + LLM summarization — February/March 2026
- **xMemory** (Alan Turing Institute / King's College London) — four-level semantic hierarchy for context compression — published as academic research in 2026
- **NeoCognition** (TechCrunch, April 21, 2026) — **$40M seed** to build "self-learning agents that build world models for any profession or environment" — direct validation of the Dewey thesis, funded at a scale that makes the convergent validation impossible to dismiss

Orca's differentiator: Dewey has been a named, running component of a shipped application since before the academic paper was published and before NeoCognition took the seed round. A librarian as a role, built by a solo developer in Eastern Kentucky, before a team of PhDs and $40M arrived at the same architecture.

---

5. Validated Transport Layer for Agent Communication (AHP / Mailman)

The concept: A dedicated transport layer for agent-to-agent communication using typed task packets (scope, constraints, expected output schema) rather than loose natural-language handoffs or Python function calls. Contracts, not conversations.

Prior art:

- `junkyard22/Mailman` — TypeScript transport implementation
- `junkyard22/AHP` — **public** specification repository
- `@marsulta/mailman` v0.1.0 — **published to npm April 7, 2026** — full runtime with middleware pipeline, retry policy, dead-letter queue, SQLite trace store, typed packet schema with lifecycle state machine, pub/sub event bus, streaming support, load balancer, ack/nack patterns, CLI, telemetry hooks

- Three-agent proof of concept verified: Orchestrator → Brain → Pappy → return to Orchestrator with full trace

What shipped later:

- **AgentMail** (Product Hunt, April 2026) — structured email-based agent handoffs — conceptually adjacent to Mailman but lacks the schema, quality gate, verdict types, and curriculum signal. Could theoretically use AHP as the smarter layer above it.
- **The New Stack** — "OpenClaw vs Hermes Agent" (April 2, 2026) — framed validated transport as an emerging category: "a validated transport layer so each role receives clear scope, constraints, and expected outputs in a predictable format"

---

6. MCP Diagnostic Tooling (Pipewrench)

The concept: A standalone tool for diagnosing and debugging MCP server connections, before MCP became widely adopted.

Prior art:

- `YakStacks/Pipewrench` — **public repository**, JavaScript, MIT License — Mar 2, 2026
- Description: "MCP Connection Diagnostic & Proxy Tool"

---

7. Local-First AI Task Runner (Workbench)

The concept: A local-first AI task runner with no cloud dependency, no subscription, built around chatting with AI to chain tools together.

Prior art:

- `YakStacks/Workbench` — **public repository**, JavaScript, MIT License — active Feb 2026
- Description: "Local-First AI Task Runner. Build automations by chatting with AI. No cloud, no subscription."

---

8. Orca as Agent Operating System (Conceptual Framing)

The concept: Orca as a personal operating system for AI agents — not a feature, not a tool, but an environment where agents run under contracts, gates, and a persistent user context. The integrated surface rather than any single component.

Prior art:

- Conversation on March 10, 2026 (Claude session, timestamped): Orca framed explicitly as a personal operating system
- All architectural components (Brain, Benson, Miranda, Pappy, Dewey, Moonshiner, Maestro, AHP) built and running prior to public "agent OS" category formation

What shipped later:

- **Qualixar OS** (arXiv:2604.06392, April 7, 2026) — feature-maximalist agent OS with 12 topologies, 24-tab dashboard, 2,821 tests, marketplace. Published 28 days after the "personal operating system" framing was applied to Orca in the timestamped Claude conversation. Qualixar uses judge-based consensus for quality (post-hoc); Orca uses Pappy gating (pre-propagation).
- **AIOS** — kernel-level resource scheduling for agents — not contract-based

The distinction: Qualixar OS built the feature surface top-down. Orca built the contract layer bottom-up. The "agent OS" category is forming publicly; Orca's counter-positioning (contract-first, quality-gated, local-first, no session tax) is sharper for having been built from the problem rather than toward the category.

Competitive Validation Timeline

Date	Event	Orca Component Validated
Nov 29, 2025	Moonshiner + moonshiner-recipes repos created	Pappy-gated distillation loop, MoonScript DSL
Dec 11, 2025	Neural Equalizer System conceived (separate framework, timestamped prior art)	—
Feb 2026	Hermes Agent launches with self-training loop	Moonshiner (no quality gate)
Feb 2026	GitHub Squad announced	Pappy + Maestro multi-agent coordination
Feb 2026	AgentMail launches	AHP / Mailman transport concept

Date	Event	Orca Component Validated
Mar 2026	MiniMax M2.7 — self-evolving capabilities	Moonshiner + jar architecture
Mar 2026	Cursor Composer 2 — self-summarization/distillation	Jar concept
Mar 2026	OpenClaw supply chain attack (CVE-2026-25253)	Miranda's tool gate as the correct answer
Mar 2026	Hermes Agent v0.6.0 — SQLite persistent memory	Dewey (predates)
Mar 2026	GitAgent — "Docker for AI agents"	Miranda + Dewey + Maestro adapter architecture
Mar 2026	xMemory (Alan Turing Institute / King's College London)	Dewey context compression
Mar 2026	Memento-Skills (arXiv:2603.18743) — "Let Agents Design Agents"	Moonshiner (no quality gate)
Mar 10, 2026	"Orca as personal operating system" framing in timestamped Claude conversation	Orca OS conceptual prior art
Apr 2, 2026	The New Stack: "OpenClaw vs Hermes Agent" — entire architecture framed as emerging category	Integrated Orca architecture
Apr 7, 2026	`@marsulta/mailman` v0.1.0 published to npm	AHP runtime public
Apr 7, 2026	Qualixar OS published (arXiv:2604.06392)	Agent OS framing (28 days after Orca's)
Apr 2026	Anthropic Claude Managed Agents launches	Miranda + Maestro as cloud platform ($0.08/session-hour cloud tax)
Apr 2026	Google Antigravity — parallel agents in IDE	Horizontal multi-agent (orthogonal to Orca's vertical contract pipeline)
Apr 2026	Qodo webinar — automated quality gates + living governance	Pappy + Moonshiner

Date	Event	Orca Component Validated
Apr 21, 2026	**NeoCognition raises $40M seed** to build self-learning specialist agents	Dewey + jar architecture
Apr 24, 2026	Claude Code screenshot — visible multi-model routing (Haiku for Bash, Opus for reasoning)	Jar pattern shipped as user-facing feature by Anthropic

---

How to Verify

All timestamps are verifiable through GitHub's public API, npm registry, arXiv, and published press:

```bash
# Verify Moonshiner creation date
curl https://api.github.com/repos/YakStacks/moonshiner-recipes

# Verify Orca commit history (local)
git -C /path/to/Orca log --pretty=format:"%h | %ad | %s" --date=short

# Verify AHP publication
curl https://api.github.com/repos/junkyard22/AHP

# Verify Mailman npm publication
curl https://registry.npmjs.org/@marsulta/mailman

# Find first commit introducing key concepts (local)
git -C /path/to/Orca log --all --oneline -S "Moonshiner"
git -C /path/to/Orca log --all --oneline -S "Pappy"
git -C /path/to/Orca log --all --oneline -S "task_graph"
git -C /path/to/Orca log --all --oneline -S "Miranda"
git -C /path/to/Orca log --all --oneline -S "Dewey"
```

The `moonshiner-recipes`, `Workbench`, `Pipewrench`, `AHP`, and `Neural-Equalizer-System-` repositories are **public** with verifiable commit history. The `@marsulta/mailman` npm package is publicly installable.

Statement

All concepts documented here were independently conceived by James Yarber, a solo developer

in Eastern Kentucky, without external funding, without a team, without a background in software engineering, and without knowledge of competing implementations at the time of creation. The pattern of well-funded teams — some with tens of millions of dollars in seed funding and teams of PhDs — shipping individual pieces of this integrated architecture after the architecture was committed to version control, is documented here for the record.

The contractual relationship between Pappy (quality verifier) and Moonshiner (distillation pipeline) — where only Pappy-verified runs enter the training loop — remains, as of April 2026, unreplicated in any public system. Every major self-improvement loop shipped by competitors (Hermes, MiniMax M2.7, Memento-Skills, Qualixar OS) lacks this gate. The Qualixar benchmark showing declining scores across iterations is direct empirical evidence of what happens without it.

The thesis: **the model is the spigot, the contract is the nozzle.** Most of the industry is selling better spigots. Orca is selling the nozzle.

Document last updated: April 24, 2026